

Exercício em duplas (10 min): qual método é chamado? (1/3)

Para cada trecho: **(a)** qual é a saída? **(b)** a decisão foi tomada na **compilação** ou na **execução**?

1 • Java

```
class Animal {  
    String som() { return "..."; }  
}  
class Cachorro extends Animal {  
    String som() { return "au"; }  
}  
Animal x = new Cachorro();  
System.out.println(x.som());
```

(a) qual é a saída?

R: 'au'.

(b) a decisão foi tomada na **compilação** ou na **execução**?

R: Foi tomada na EXECUÇÃO, ao escolher pelo tipo do Objeto, que foi Cachorro.

2 • C++

```
struct A {  
    void f() { cout << "A"; }  
};  
struct B : A {  
    void f() { cout << "B"; }  
};  
B b; A *p = &b;  
p->f();
```

(a) qual é a saída?

R: 'A'.

(b) a decisão foi tomada na **compilação** ou na **execução**?

R: Foi tomada na COMPILAÇÃO, decidida pelo ponteiro A* por conta de não ser “virtual”. Se tivesse o método como “virtual void”, passaria a ser dinâmico e ser decidido na execução.

Exercício: qual método é chamado? (2/3)

Para cada trecho: **(a)** qual é a saída? **(b)** a decisão foi tomada na **compilação** ou na **execução**?

3 • Java

```
class A { String nome = "A";  
    String getNome() { return nome; } }  
class B extends A { String nome = "B";  
    String getNome() { return nome; } }  
A x = new B();  
System.out.println(x.nome + " " + x.getNome());
```

(a) qual é a saída?

R: 'A B'.

(b) a decisão foi tomada na **compilação** ou na **execução**?

R: Teve ambos os casos. O 'x.nome' foi uma decisão tomada em COMPILAÇÃO, pois atributos em Java não são polimórficos, enquanto o uso do método get no 'x.getNome()' tem sua decisão tomada em EXECUÇÃO, visto que métodos em Java são polimórficos.

4 • Python

```
class Contador:  
    total = 0  
    def __init__(self):  
        Contador.total += 1  
        self.id = Contador.total
```

```
a = Contador(); b = Contador()
```

```
print(a.id, b.id, a.total)
```

(a) qual é a saída?

R: '1 2 2'.

(b) a decisão foi tomada na **compilação** ou na **execução**?

R: Foi tomada na EXECUÇÃO, pois o Python é uma linguagem dinamicamente interpretada, ou seja, métodos, atributos, classes são decididas apenas em tempo de execução.

Exercício: qual método é chamado? (3/3)

Para cada trecho: **(a)** qual é a saída? **(b)** a decisão foi tomada na **compilação** ou na **execução**?

5 • Java

```
class A {  
    static String quem() { return "A"; }  
}  
class B extends A {  
    static String quem() { return "B"; }  
}  
A x = new B();  
System.out.println(x.quem());
```

(a) qual é a saída?

R: 'A'.

(b) a decisão foi tomada na **compilação** ou na **execução**?

R: Foi tomada na COMPILAÇÃO, pois o método é static, e métodos estáticos em Java não são polimórficos, logo, a decisão delas não é tomada em tempo de execução, e sim compilação.

6 • Go

```
type Animal struct{}  
func (Animal) Som() string { return "... " }
```

```
func (a Animal) Falar() string {  
    return "faz " + a.Som()  
}  
type Cao struct{ Animal }  
func (Cao) Som() string { return "au" }  
  
fmt.Println(Cao{}.Falar(), Cao{}.Som())
```

(a) qual é a saída?

R: 'faz ... au'.

(b) a decisão foi tomada na **compilação** ou na **execução**?

R: Foi tomada na COMPILAÇÃO, pois GO não é dinamicamente interpretado, o completo oposto de Python. Logo, as decisões são tomadas em tempo de compilação.